

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра информационных технологий

КУРСОВОЙ ПРОЕКТ
ГЕНЕРАЦИЯ СИНТАКСИЧЕСКИХ И ЛОГИЧЕСКИХ ОШИБОК В
ИСХОДНОМ КОДЕ НА ОСНОВЕ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ

Работу выполнил _____
(подпись)

Направление подготовки 02.03.03 Математическое обеспечение и администрирование информационных систем

Направленность (профиль) Технологии разработки программных систем

Научный руководитель

.... _____
(подпись, дата)

Нормоконтролер

... _____
(подпись, дата)

Краснодар
2026

Пример паспорта проекта

Название проекта	ГЕНЕРАЦИЯ СИНТАКСИЧЕСКИХ И ЛОГИЧЕСКИХ ОШИБОК В ИСХОДНОМ КОДЕ НА ОСНОВЕ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ
-------------------------	---

Команда проекта			
№	Ф.И.О.	Роль	Функциональные обязанности в ходе реализации проекта и компетентность
1.	Тополян Алина Варткесовна	Fullstack разработчик, ведущий программист	Разработка архитектуры ПС и написание кода на Python, Kotlin, тестирование ПС

1. География проекта	Краснодарский край, г. Краснодар
	<i>перечислить субъекты РФ, районы и населенные пункты, на которые распространяется проект</i>

2. Сроки реализации проекта	Начало реализации	Окончание реализации
	Февраль, 2026 (месяц, год)	Май, 2026 (месяц, год)

3. Краткая аннотация проекта	Курсовой проект посвящен проектированию и разработке серверного компонента на платформе Django для мобильного обучающего приложения по программированию. Сервер обеспечивает автоматическую генерацию учебных заданий на поиск и исправление синтаксических и логических ошибок в коде. Выполнен анализ существующих образовательных платформ и методов автоматического внесения типовых ошибок. В практической части представлена разработка REST API на Django с использованием Django REST Framework. Основные технические решения включают: нормализацию исходного кода, применение регулярных выражений для поиска синтаксических конструкций (циклы, условия, операторы), автоматическое внесение ошибок с сохранением эталонной версии, а также алгоритм сравнения исправленного пользователем кода
-------------------------------------	--

	с оригиналом. Поддерживаются языки Python, Java и JavaScript. Результатом работы является полноценный Django-сервер, обеспечивающий хранение заданий, генерацию новых упражнений на основе загруженного кода и поддержку нескольких языков программирования. Интеграция с готовым Kotlin-приложением позволила создать обучающую систему, направленную на развитие навыков анализа, отладки и сопровождения программного кода.
--	--

4. Описание проблемы, решению/снижению остроты которой посвящен проект Актуальность проекта	В условиях стремительного развития информационных технологий возрастает потребность в специалистах, обладающих не только знаниями синтаксиса языков программирования, но и устойчивыми навыками анализа, отладки и сопровождения программного кода. Практика показывает, что в профессиональной деятельности разработчик значительную часть времени тратит на поиск, локализацию и исправление ошибок в существующих программных модулях, а не написание свежего кода. Поэтому умению находить и нейтрализовать ошибки нужно уделять достаточное количество внимания во время обучения. К сожалению, большинство современных образовательных платформ ориентированы преимущественно на выполнение задач по написанию нового кода и автоматическую проверку корректности результата, уделяя недостаточное внимание формированию компетенций в области анализа и исправления ошибок. С другой стороны, разработка достаточного числа примеров кода, содержащих синтаксические и логические ошибки, требует от преподавателя трудоемких ресурсов. Необходим инструмент автоматизации этого процесса. В связи с этим актуальной является разработка обучающей системы, ориентированной на формирование навыков выявления и устранения типовых синтаксических и логических ошибок, включающей функцию автоматической генерации заданий для анализа.
---	--

5. Основные целевые группы, на которые направлен проект	Студенты IT-специальностей : Потребности и проблемы: необходимость формирования навыков отладки и анализа кода, востребованных на рынке труда, ограниченное количество практических заданий на поиск и исправление ошибок в рамках учебных курсов, отсутствие удобного мобильного инструмента для самостоятельной тренировки вне аудитории. Проект даст им возможность тренироваться на неисправном коде, приближенном к реальным задачам и обеспечит мобильным доступ к заданиям в любое время. Преподаватели дисциплин, связанных с программированием: Потребности и проблемы: высокие временные затраты на подготовку качественных заданий по поиску ошибок, включающий в себя потребность в придумывании кода, ввода ошибок вручную, подготовка эталонного решения. Ограниченный набор готовых заданий, сложность распределения для разных групп студентов. Необходимость адаптирования заданий под конкретные темы и язык программирования. Проект решает эти задачи путем автоматической генерации заданий ошибочных фрагментов кода, возможностью выбора сложности заданий и поддержкой различных языков программирования. Самостоятельные пользователи и начинающие разработчики: Потребности и проблемы: отсутствие систематической практики по отладке кода, сложность найти качественные задания на исправление ошибок. Проект решает задачи, предоставляя готовые задания, загруженные другими пользователями, из базы данных, а также тем, что дает доступный инструмент автоматической проверки введенных пользователями ответов.
6. Основная цель проекта	Проектирование и разработка серверного модуля, обеспечивающего автоматическую генерацию заданий с синтаксическими и логическими ошибками в исходном коде с использованием регулярных выражений, и его интеграция с мобильным приложением на Kotlin.

7. Задачи проекта	1. Выполнить сравнительный анализ существующих программных решений для автоматизации генерации заданий 2. Разработать функциональную структуру программного продукта 3. Определить функциональные и нефункциональные требования 4. Разработать архитектуру серверного модуля с генерацией заданий на нахождение ошибок 5. Предложить классификацию генерируемых ошибок 6. Разработать набор регулярных выражений для парсинга кода 7. Реализовать модуль нормализации исходного кода 8. Реализовать модуль поиска и модификации кода на основе regex 9. Разработать API для образовательного приложения по программированию, включающее модуль генерации заданий с ошибками 10. Реализовать механизм верификации ответа пользователя 11. Реализовать графический интерфейс для поиска ошибок в коде 12. Выполнить интеграцию серверного модуля с существующим мобильным приложением на Kotlin 13. Выполнить тестирование разработанного модуля для генерации ошибок в коде. 14. Подготовить материалы к защите курсового проекта
--------------------------	---

8. Календарный план реализации проекта			
Решаемая задача	Метод/мероприятие и его описание	Сроки (дд.мм.гг)	Показатели результативности
Задача 1. Выполнить сравнительный анализ существующих программных решений для автоматизации генерации заданий	Сравнительный анализ, классификация, метод экспертных оценок, анализ документации и научных публикаций	2.02.26-14.02.26	Количество проанализированных аналогов (не менее 3); выявление не менее 3 критериев, по которым аналоги уступают разработанному решению
Задача 2. Разработать	Объектно-ориентированное	15.02.26 – 28.02.26	Разработана функциональная

функциональную структуру программного продукта	проектирование, построение диаграмм вариантов использования (Use Case) и функциональных диаграмм		схема системы, выделено не менее 3 основных автоматизированных функций
Задача 3. Определить функциональные и нефункциональные требования	Анализ пользовательских сценариев	01.03.26-07.03.26	Сформулировано не менее 10 функциональных требований (поддержка языков, типы ошибок, уровни сложности) и не менее 5 нефункциональных.
Задача 4. Спроектировать базу данных для программной системы	Построение ER-диаграммы	08.03.26-20.03.26	Спроектирована схема базы данных (не менее 4 сущностей)
Задача 5. Предложить классификацию генерируемых ошибок	Анализ типовых ошибок начинающих программистов, таксономия, метод категорирования	21.03.26-27.03.26	Разработана классификация, включающая не менее 3 категорий и не менее 10 конкретных типов ошибок
Задача 6. Разработать набор регулярных выражений для парсинга кода	Теория формальных языков, синтаксический анализ, разработка и тестирование regex-шаблонов на контрольных примерах кода	28.03.26-5.04.26	Разработано не менее 15 регулярных выражений для поиска конструкций в Python, Java, JavaScript. Точность обнаружения целевых конструкций не менее 90%
Задача 7. Реализовать модуль нормализации исходного кода	Рефакторинг кода, обработка строк, приведение отступов и переносов к единому формату	6.04.26-09.04.26	Модуль обеспечивает нормализацию кода на 3 языках. Время нормализации

			фрагмента кода (до 100 строк) не более 0,5 секунды
Задача 8. Реализовать модуль поиска и модификации кода на основе regex	Алгоритмизация процесса внесения ошибок, случайный выбор фрагментов, применение преобразований с сохранением эталона	10.04.26-17.04.26	Модуль поддерживает внесение не менее 10 типов ошибок. Количество успешных модификаций на 100 тестовых примерах не менее 90%. Сохранение эталонной версии в 100% случаев
Задача 9. Разработать API для образовательного приложения по программированию, включающее модуль генерации заданий с ошибками	Проектирование REST API, реализация эндпоинтов	18.04.26-01.05.26	Реализовано не менее 5 REST-эндпоинтов. Время ответа API на запрос не более 1 секунды
Задача 10. Реализовать механизм верификации ответа пользователя	Нормализация перед сравнением, подсветка расхождений построчно и посимвольно	02.05.26 – 03.05.26	Точность верификации (совпадение с эталоном) не менее 98%. Время проверки одного ответа не более 0,5 секунды. Подсветка несоответствий реализована на уровне строк и символов
Задача 11. Реализовать графический интерфейс для поиска ошибок в коде	Верстка интерфейса для выполнения заданий в мобильном приложении на Android Kotlin	04.05.26-08.05.26	Разработан экран мобильного приложения для работы с заданием. Время освоения интерфейса новым

			пользователем не более 5 минут
Задача 12. Выполнить интеграцию серверного модуля с существующим мобильным приложением на Kotlin	Настройка HTTP-клиента в Kotlin (Retrofit/OkHttp), обработка JSON-ответов, синхронизация данных между локальной БД и серверной.	09.05.26-12.05.26	Обеспечена полная интеграция: приложение успешно получает задания с сервера, отправляет ответы и получает результаты верификации. Данные между локальной и серверной БД успешно синхронизируются в 99% случаев.
Задача 13. Выполнить тестирование разработанного модуля для генерации ошибок в коде	Модульное тестирование (unit tests), функциональное тестирование, тестирование на контрольных примерах	13.05.26-15.05.26	Доля успешно пройденных модульных тестов не менее 90%. Количество автоматизированных тестов не менее 10. Отсутствие критических ошибок
Задача 14. Подготовить материалы к защите курсового проекта	Подготовка презентации, реферирование работы	12.05-15.05	Доклад и презентация

9. Показатели результативности реализации проекта:

Количественные показатели

Аналитическая часть:

- проведен анализ не менее 3 существующих программных аналогов,
- выявлено не менее 3 функциональных недостатков существующих решений,
- сформулировано не менее 10 функциональных требований,
- сформулировано не менее 5 нефункциональных требований,

	– логическая модель БД (не менее 4 сущностей). Проектная часть: – физическая модель БД, – UML-диаграммы (классов, компонентов, развертывания). Реализационная часть: – наполнение БД (не менее 100 записей), – рабочее мобильное приложение, – тестовые сценарии.
Качественные показатели	Пользовательские характеристики: – интуитивно понятный интерфейс, не требующий специального обучения, – время освоения интерфейса новым пользователем: не более 5 минут, – эргономичность расположения элементов управления, – визуальная обратная связь на действия пользователя. Качество обработки данных: – корректность применения правил ГОСТ Р 7.0.100-2018 к различным типам источников – сохранение смысловой целостности библиографических записей при конвертации, – отсутствие потери значимых элементов описания, Надежность и стабильность: – отсутствие критических ошибок (класс severity 1), – устойчивость к некорректному вводу данных (обработка исключительных ситуаций), – корректное завершение работы при возникновении ошибок. Юзабилити: – субъективная оценка удобства интерфейса целевыми пользователями: не ниже 4 баллов из 5, – логичность последовательности действий при работе с сервисом, – наличие понятных подсказок и сообщений об ошибках.

10. Партнеры проекта и собственный вклад		
№	Партнер/ Собственный вклад	Вид поддержки
1.	Факультет компьютерных технологий и прикладной математики	информационная
2.	Кафедра информационных технологий	консультационная
3.	Собственный вклад	ресурсная (ноутбук, специальное ПО)

11. Дальнейшая реализация и мультипликативность проекта
Расширение функциональных возможностей: – добавление языков C++, C#, Go, TypeScript с соответствующими наборами регулярных выражений, – добавление импорта кода с Github/Gitlab, – реализация взаимодействия между пользователями, отправление набора заданий или кода, – реализация дополнительных типов ошибок. Технологическое развитие: – развёртывание на сервере, – создание веб версии приложения.

12. Информационное сопровождение проекта	
Планируемое информационное освещение проекта в СМИ	Наименование и характеристика СМИ (форма, тираж, охват аудитории)
	Размещение исходного кода в репозитории Github
	Опубликование статьи ... в сборнике....

13. Приложения и дополнительная информация о проекте
Отчет о реализации проекта Исходный код проекта: https://github.com/....

14. Детализированная смета проекта					
№ п/п	Статья расходов	Стоимость	кол-во	Итого	Обоснование и источник
1.	Доступ к научным базам данных	бесплатно			

2.	Приобретение ПО для разработки	бесплатно			
3.	Формирование тестовых наборов данных				Личный вклад

Отчет о реализации проекта

1 Сравнительный анализ генерации учебных заданий

Для проведения сравнительного анализа были выбраны следующие программные продукты, представляющие различные подходы к генерации учебных заданий и работе с ошибками в коде:

- LeetCode;
- Stepik;
- MutPy.

LeetCode – это онлайн-платформа для подготовки к техническим собеседованиям в IT-компаниях, развития навыков решения алгоритмических задач и улучшения качества кода. Она предоставляет коллекцию задач по алгоритмам, структурам данных и системному дизайну, которые помогают программистам совершенствовать свои умения.

Платформа предоставляет широкий выбор языков программирования и качественные задания от ведущих IT-компаний, а встроенные редакторы кода позволяют тестировать решения сразу. Платформа автоматически оценивает код на корректность и эффективность. К недостаткам можно отнести то, что задания на совершенствование навыков отладки кода создаются авторами вручную, автоматическая генерация отсутствует, потому что основной фокус заданий платформы на алгоритмы, а не отладку. Полноценный доступ к заданиям платный [11].

Stepik – российская образовательная платформа и конструктор онлайн-курсов. Она позволяет пользователям учиться и создавать собственные образовательные программы. Она бесплатная, предоставляет поддержку множества языков программирования и возможность создавать и загружать свои задания. Но также, как и LeetCode, не лишена недостатков: задания на поиск ошибок создаются преподавателями вручную, проверка осуществляется

только через тесты, без локализации конкретных ошибок в коде пользователя [12].

Говоря о генерации некорректного кода, необходимо также упомянуть инструменты мутаций, например такие как MutPy. MutPy – это инструмент мутационного тестирования для языка Python, который автоматически вносит небольшие изменения (мутации) в исходный код с целью проверки качества набора тестов. Если тест проходит на измененном коде – значит тест написан плохо.

В отличие от других рассматриваемых решений MutPy – это только о генерации измененных версий кода, а не об обучении отладке. Он предназначен для профессионального тестирования, у него отсутствует графический интерфейс, мутации не классифицированы по уровням сложности и отсутствует поддержка языков, кроме Python [6].

Проведённый сравнительный анализ показал, что существующие программные решения не обеспечивают комплексного подхода к автоматизации генерации учебных заданий на поиск и исправление ошибок в программном коде. LeetCode и Stepik, являясь популярными образовательными платформами, требуют ручного создания заданий на отладку и не поддерживают адаптивность по сложности. MutPy демонстрирует возможность автоматической модификации кода, но создан для профессионального тестирования, не имеет удобного интерфейса и не адаптирован для обучения.

Таким образом, актуальность разработки собственного модуля генерации синтаксических и логических ошибок на основе регулярных выражений, интегрированного с мобильным обучающим приложением, обоснована выявленными недостатками существующих аналогов. Разрабатываемое решение позволит автоматизировать процесс создания заданий на отладку, обеспечить адаптивность обучения и сделать тренировку навыков анализа кода доступной в мобильном формате.

2 Функциональная структура программной системы

В рамках задачи разработки функциональной структуры программной системы построена функциональная модель разрабатываемой ПС в виде контекстной диаграммы в нотации IDEF0, которая изображена на рисунке 1.

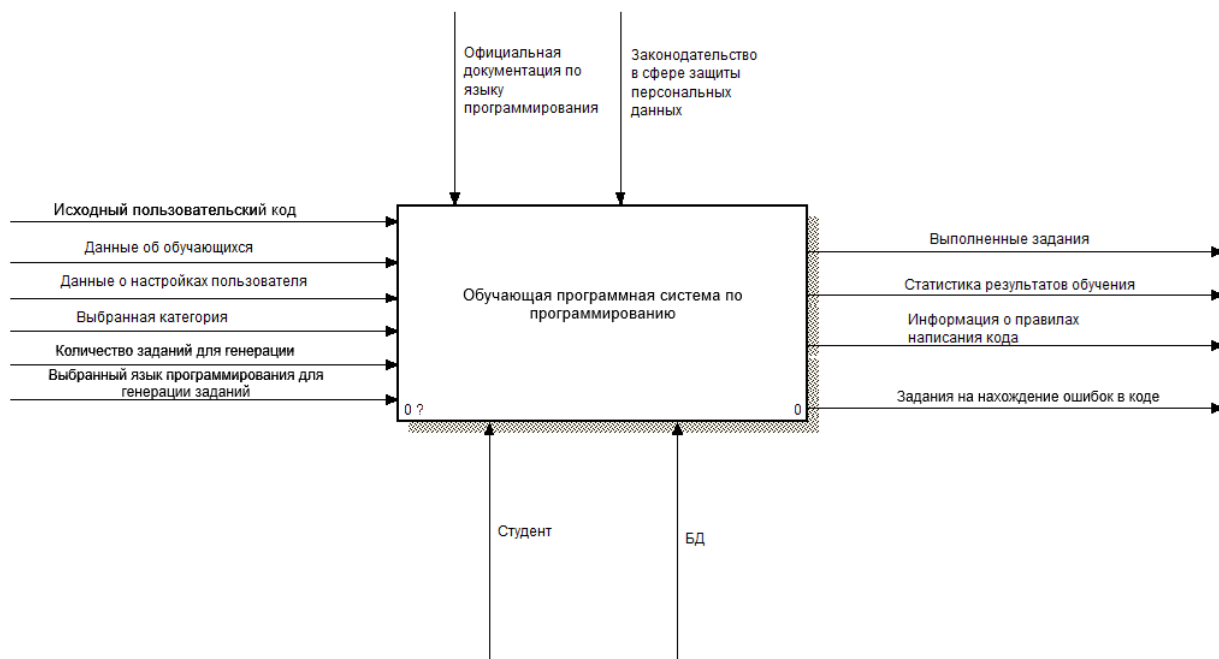


Рисунок 1 – Контекстная диаграмма IDEF0

На контекстной диаграмме определены границы системы, её основные входы, выходы, механизмы и управляющие воздействия. Входными данными системы являются: исходный пользовательский код, данные об обучающихся, настройки пользователя, выбранная категория ошибок, количество заданий для генерации и выбранный язык программирования. Управляющими воздействиями выступают официальная документация по языку программирования и законодательство в сфере защиты персональных данных. Механизмами системы являются студент и база данных. Выходными результатами системы выступают сгенерированные задания на нахождение ошибок в коде, выполненные задания, статистика результатов обучения и информация о правилах написания кода.

На рисунке 2 представлена Use Case диаграмма разрабатываемой системы, отражающая функциональные требования с точки зрения внешних пользователей.

Основным эктором является студент, который взаимодействует с системой через мобильное приложение. Он имеет возможность зарегистрироваться, войти в систему, проходить квизы с простыми тестовыми вопросами и квизы с практическими заданиями. Без авторизации прохождение квизов, выставление пользовательских настроек и просмотр статистики невозможны. Более подробно описано на диаграмме.

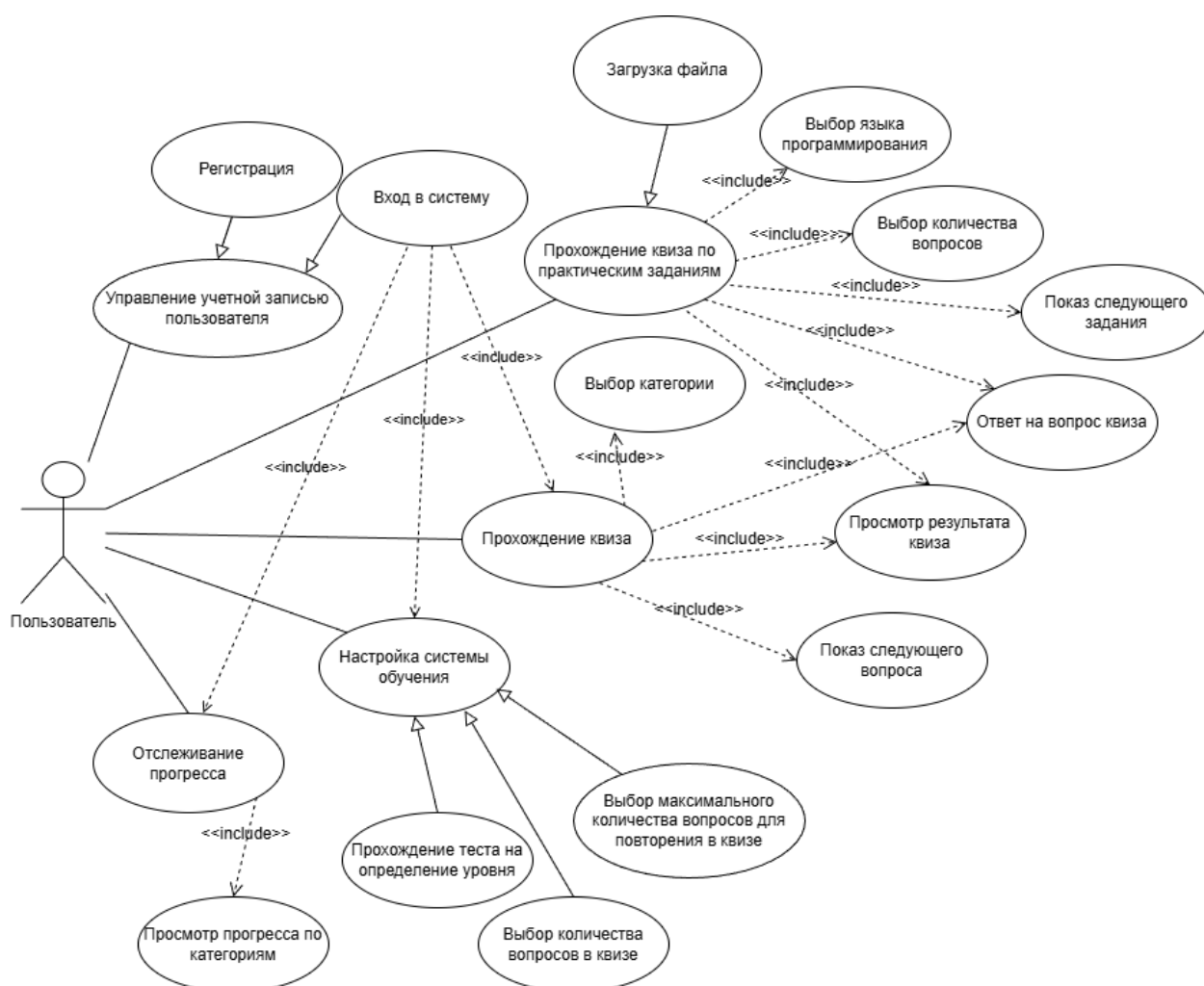


Рисунок 2 – Use Case диаграмма ПС

Говоря об алгоритме генерации ошибок в коде, можно выделить три основные функции: загрузка пользовательского файла с устройства, парсинг

кода, т.е. выделение отдельных блоков, которые могут рассматриваться как семантически законченные и осмысленные, а также применение мутаций с помощью регулярных выражений. Функциональная схема этой части системы отображена на рисунке 3.

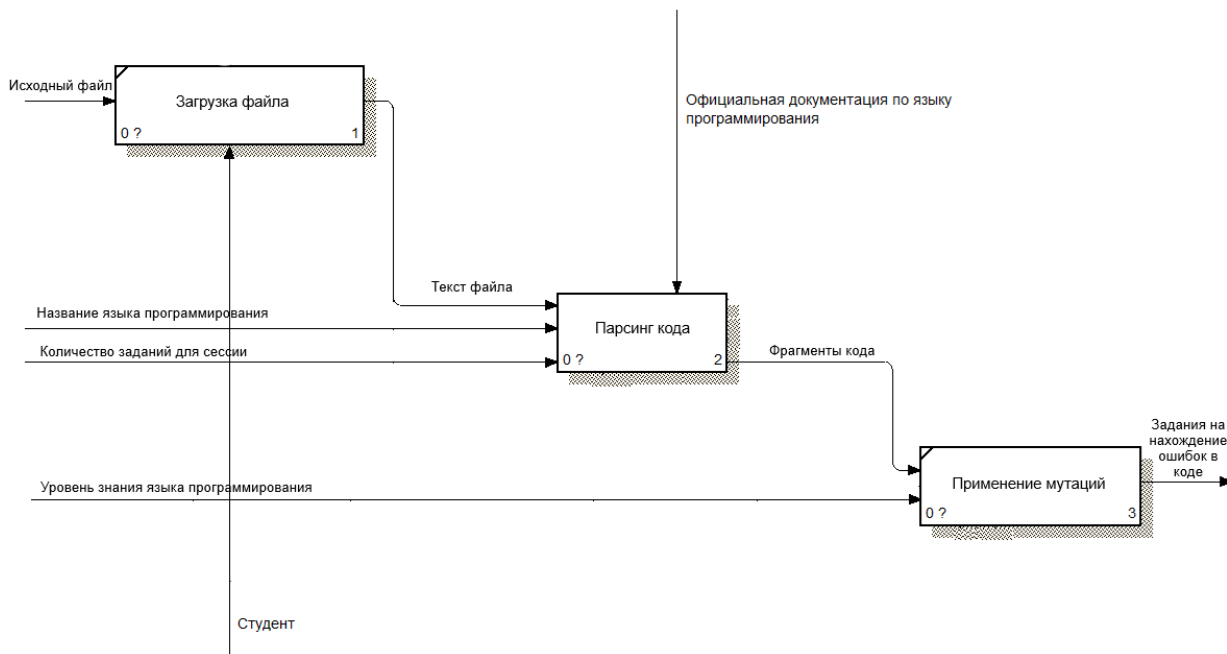


Рисунок 3 – Функциональная схема алгоритма генерации ошибок

Построенные диаграммы (IDEF0, Use Case, функциональная схема) полностью описывают функциональную структуру системы: определены границы, внешние пользователи, основные входы и выходы, а также ключевые функции генерации ошибок — загрузка файла, парсинг кода и применение мутаций.

3 Функциональные и нефункциональные требования

В ходе анализа пользовательских сценариев и выделенных ранее вариантов использования сформулированы функциональные и нефункциональные требования, предъявляемые к модулю генерации ошибок, а также к связанному с ним новому функционалу мобильного приложения. Данные требования описаны далее.

Рассмотрим задачу «Прохождение квиза по практическим заданиям». Пользователь должен иметь возможность загрузить файл, по которому будет решать задания в текущей сессии обучения. Также должна быть предусмотрена возможность прохождения сессии без загрузки файла. В таком случае система будет предоставлять пользователю уже имеющиеся задания из базы данных.

К подзадаче «Загрузка файла» сформулированы следующие функциональные требования:

- 1) пользователь должен иметь возможность загружать файлы с расширениями: *.py, *.txt, *.js, *.java;
- 2) система должна принимать файлы только допустимого расширения;
- 3) загруженный файл должен сохраняться на сервере с уникальным именем для предотвращения конфликта имён;
- 4) если расширение файла не соответствует допустимому, должно отображаться сообщение об ошибке;
- 5) если загружаемый файл пуст, система должна выводить сообщение об ошибке;
- 6) минимальная длина файла – 100 строк.

Рассмотрим требования к действиям программы для подзадачи «Выбор языка программирования»:

- 1) пользователь должен иметь возможность выбрать язык программирования для сессии из выпадающего списка;
- 2) языки программирования берутся из базы данных;
- 3) должны быть доступны следующие языки программирования: Java, JavaScript, Python.

Рассмотрим требования к действиям программы для подзадачи «Выбор количества заданий»:

- 1) пользователь должен иметь возможность выбрать количество заданий для текущей сессии;

- 2) минимальное количество заданий – 3, максимальное – 30.

Рассмотрим требования к действиям программы для подзадач «Показ следующего задания» и «Ответ на вопрос квиса»:

Пользователь должен увидеть текст следующего задания, количество правильных и неправильных ответов, фрагмент кода в изменяемом поле и кнопку для проверки задания. Прямо в это поле пользователь должен вносить исправления. После нажатия на кнопку в зависимости от правильности решения должно изменяться количество правильных и неправильных ответов. Если пользователь ответил верно, поле должно подсвечиваться зелёным цветом, а изменённая строка — жирным зелёным шрифтом. Если пользователь ответил на задание неверно, должно появиться дополнительное поле, подсвеченное зелёным, в котором зелёным жирным шрифтом выделена строка, содержащая ошибку. Старое поле должно окрашиваться в красный цвет. После проверки кнопка должна меняться на кнопку «Далее», при нажатии на которую будет осуществляться переход к следующему заданию.

Рассмотрим требования к действиям программы для подзадачи «Просмотр результатов квиса»:

Как только пользователь решит все задания, должен отобразиться экран с результатами квиса. На этом экране пользователь должен увидеть количество правильных и неправильных ответов, процент правильных ответов и общее количество вопросов. Также должна отображаться крупная кнопка возврата в главное меню.

К алгоритму генерации ошибок были сформулированы следующие функциональные требования:

- 1) алгоритм должен генерировать задания для трёх языков программирования: Python, Java, JavaScript;
- 2) модуль должен автоматически вносить синтаксические и логические ошибки во фрагменты кода пользователя;

- 3) алгоритм должен делить код на количество фрагментов, соответствующее количеству заданий, которое указал пользователь, преимущественно выделяя логически законченные блоки;
- 4) каждое задание не должно быть длиннее 15 строк;
- 5) мутации, вносимые в код, должны быть классифицированы по уровням сложности: лёгкий, средний, сложный;
- 6) при генерации заданий алгоритм должен учитывать уровень подготовки конкретного пользователя;
- 7) все сгенерированные задания должны сохраняться в базу данных вместе с эталонной версией.

Рассмотрим нефункциональные требования, предъявляемые к разрабатываемому модулю генерации ошибок и связанному с ним новому функционалу мобильного приложения.

Сформулировано требование к производительности генерации заданий: время генерации заданий (разбиение кода на фрагменты, внесение мутаций, сохранение в базу данных) не должно превышать 1 секунды на одно задание.

Выделено требование к производительности верификации ответов: время проверки ответа пользователя (сравнение исправленного кода с эталонной версией, формирование подсветки несоответствий) не должно превышать 0,5 секунды.

К производительности интерфейса были сформулированы следующие требования:

- переход между заданиями и загрузка экрана с результатами должны происходить без заметных задержек для пользователя;
- время отклика интерфейса не должно превышать 0,3 секунды.

К масштабируемости и расширяемости выделены требования:

- архитектура модуля должна позволять добавлять новые языки программирования без изменения основного кода;
- должна быть предусмотрена возможность добавления новых типов мутаций (ошибок) без модификации ядра алгоритма.

Требования к удобству использования:

- сообщения об ошибках при загрузке файла должны быть понятны пользователю и содержать рекомендации по исправлению;
- подсветка правильных и неправильных ответов должна быть наглядной и не требовать дополнительных пояснений;
- время освоения интерфейса новым пользователем не должно превышать 5 минут.

4 Проектирование БД

Для разрабатываемой программной системы была спроектирована база данных на базе PostgreSQL, список разработанных таблиц представлен в таблице 1.

Таблица 1 – Список разработанных таблиц БД

№ п/п	Имя таблицы	Описание
1	categories	Хранит данные о категориях: id категории, id языка, название категории
2	lang	Хранит данные о языках: id языка, название языка
3	options	Хранит данные о вариантах ответов: id, текст варианта ответа, правильность ответа, id вопроса
4	quiz_question	Хранит данные о вопросах: id, текст вопроса, сложность вопроса, id категории
5	srs_tools	Хранит параметры алгоритма SRS: id, фактор легкости (EF), количество повторений вопроса, дата последнего отображения, интервал, id вопроса, id пользователя
6	users	Хранит данные о пользователе: id, имя пользователя, почта, пароль
7	user_settings	Хранит данные о настройках пользователей: id, id пользователя, уровень для каждого языка, максимальное количество вопросов для повторения, количество новых вопросов в квизе
8	mistakes_algorithm_generatedtask	Хранит данные о заданиях: id задания, текст задания, фрагмент кода, строка в которой применена мутация, правильный ответ, id файла с которого сгенерировано задание, сложность задания
9	mistakes_algorithm_usercode	Хранит данные о загруженных пользователями файлах: id файла, название файла, время создания файла, язык программирования

Описание связей между таблицами представлено в таблице 2.

Таблица 2 – Связи между таблицами БД

Родительская таблица		Дочерняя таблица		Тип связи
Название	Атрибут PK	Название	Атрибут FK	
users	id	user_settings	user_id	1:M
users	id	srs_tools	user_id	1:M
lang	id	user_settings	lang_id	1:M
lang	id	categories	lang_id	1:M
categories	id	quiz_questions	category_id	1:M
quiz_questions	id	srs_tools	question_id	1:M
quiz_questions	id	options	quiz_question_id	1:M
users	id	mistakes_algorithm_usercode	user_id	1:M
langs	id	mistakes_algorithm_usercode	language_id	1:M
mistakes_algorithm_usercode	id	mistakes_algorithm_generatedtask	parent_code	1:M

ER диаграмма отображена на рисунке 4.

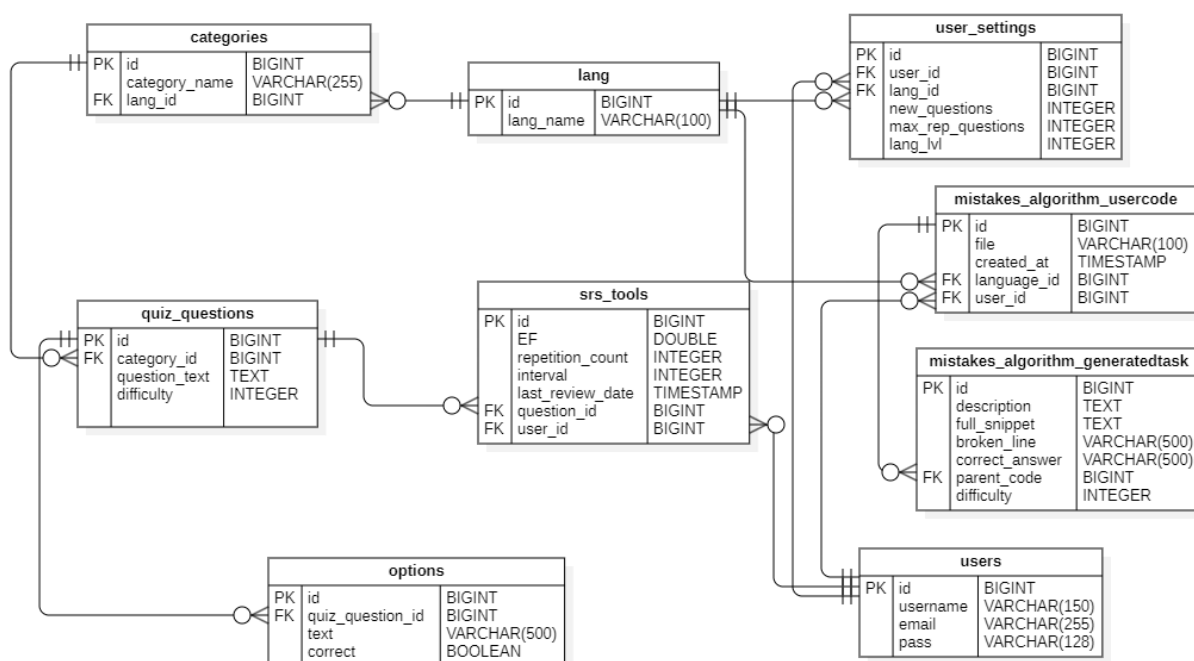


Рисунок 4 – ER-диаграмма

Представленная структура таблиц и связей между ними формирует надежную основу для интеграции с уже существующей локальной базой

данных sqlite и для хранения данных, необходимых для модуля генерации ошибок.

5 Классификация генерируемых ошибок

В системе предусмотрено 3 категории ошибок: синтаксические ошибки, логические ошибки и семантические и контекстные ошибки. Каждая ошибка отнесена к одному из трёх уровней сложности: лёгкий, средний или сложный. Это позволяет адаптировать задания под уровень подготовки обучающегося. Легкий уровень сложности подразумевает, что ошибки можно обнаружить визуально или при базовом знании синтаксиса языка программирования. Средний уровень предполагает что нахождение ошибок требует понимания логики выполнения программы. Сложный уровень – это ошибки, требующие анализа контекста, области видимости и семантики программного кода.

Синтаксические ошибки – это нарушение формальных правил языка программирования. Такие ошибки приводят к невозможности компиляции или интерпретации программы. К ним относятся:

- пропуск или добавление лишней скобки – легкий уровень сложности;
- отсутствие обязательного синтаксического элемента (например, символа «;» или «:») – легкий уровень сложности;
- ошибка в написании ключевых слов языка программирования – легкий уровень сложности;
- незакрытые строковые литералы – легкий уровень сложности;
- нарушение структуры блока (например, некорректные отступы в языке Python) – средний уровень сложности;
- некорректное объявление управляющих конструкций (if, for, while) – средний уровень сложности.

Логические ошибки не нарушают синтаксис программы, однако изменяют её поведение. Формирование подобных ошибок направлено на

развитие аналитического мышления и навыков тестирования. К ним относятся:

- неверное использование операторов сравнения – легкий уровень сложности;
- ошибки в логических операциях – средний уровень сложности;
- неверное условие завершения цикла (в том числе бесконечные циклы) – средний уровень сложности;
- некорректное использование операторов присваивания с накоплением (+, -= и др.) – средний уровень сложности.
- ложные пустые операторы циклов и условий – сложный уровень;
- ошибки в границах цикла – сложный уровень.

Формирование подобных ошибок направлено на развитие аналитического мышления и навыков тестирования.

Семантические и контекстные ошибки — ошибки, связанные с некорректным использованием переменных, функций и других сущностей программы, а также с нарушением области видимости и контекста выполнения. К ним относятся:

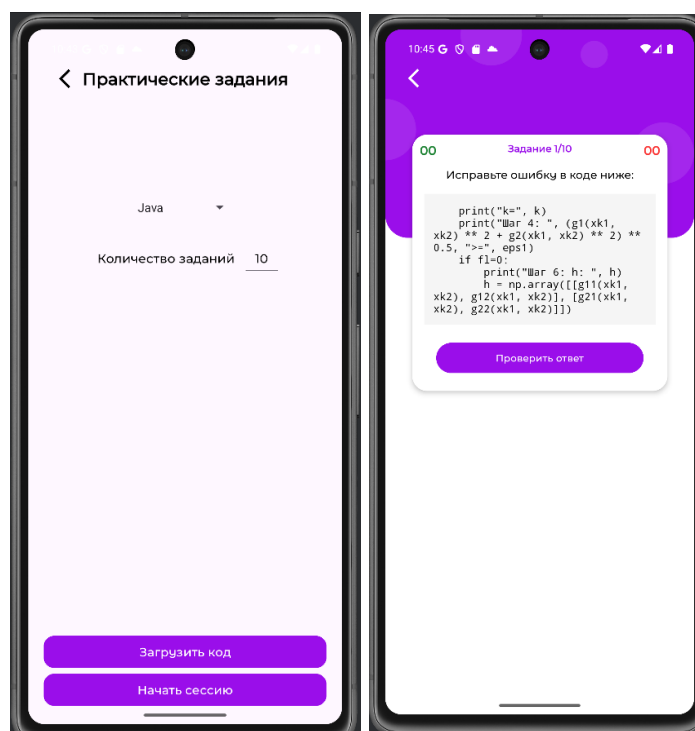
- несоответствие типов данных – средний уровень сложности;
- некорректный вызов функции (неверное количество или тип аргументов) – средний уровень сложности;
- нарушение области видимости переменной – сложный уровень;
- ошибки использования встроенных функций – сложный уровень.

Разработанная классификация учитывает, что пользователь работает с ограниченными фрагментами кода (до 15 строк), что накладывает следующие ограничения: ошибки должны быть локализуемыми в пределах одного фрагмента, исправление ошибки должно быть возможно без обращения к внешнему контексту, каждая ошибка должна иметь однозначное корректное решение.

6 Интерфейс программной системы

Для решения задач на поиск ошибок был разработан графический интерфейс в мобильном приложении.

На главном экране приложения была добавлена кнопка, перенаправляющая на экран меню для сессии практических заданий. Это меню представлено на рисунке 5а. Здесь пользователь может выбрать количество заданий для сессии, язык программирования, загрузить файл и по кнопке начать сессию.



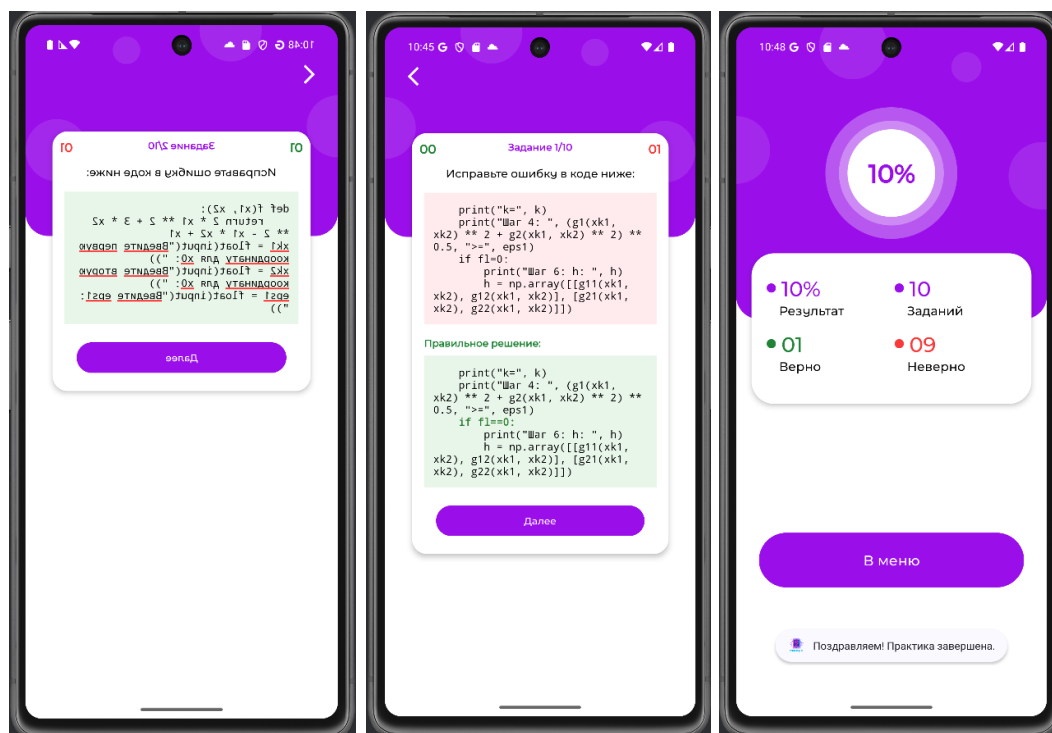
а)

б)

Рисунок 5 – Элементы интерфейса: а) главный экран; б) экран меню

После нажатия на кнопку «Начать сессию», на экране появляется первое задание (рисунок 5б). Прямо в поле с отобразившимся кодом пользователь может вносить изменения. После нажатия на кнопку «Проверить ответ» система проверяет ответ пользователя и отображает соответствующий фидбэк. Если ответ верный – поле окрашивается в зеленый, а измененная строка подсвечивается. Это отображено на рисунке 6а. Количество правильных

ответов также увеличивается. Если пользователь ответил неверно, то появляется дополнительное поле, подсвеченное зеленым, с эталонным ответом. Исходное поле окрашивается в красный (рисунок 6б). Количество неправильных ответов увеличивается. Чтобы перейти к следующему заданию пользователь должен нажать на кнопку «Далее».



а)

б)

в)

Рисунок 6 – Элементы интерфейса: а) главный экран; б) экран меню; в) экран квиза

После того как все задания были решены, отображается экран с результатами сессии, представленный на рисунке 6в.

Представленные экраны позволяют проходить квизы с практическими заданиями на поиск ошибок, обеспечивая высокий уровень интуитивности и удобства использования.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была достигнута основная цель – спроектирован и реализован серверный модуль для автоматической генерации синтаксических и логических ошибок в исходном коде с использованием регулярных выражений, интегрированный с мобильным приложением.

В процессе работы были успешно решены следующие задачи: проведён сравнительный анализ существующих решений (LeetCode, Stepik, MutPy) и выявлены их недостатки; разработана функциональная структура программной системы и определены требования к ней; спроектирована архитектура серверного модуля и база данных; предложена классификация генерируемых ошибок; разработан набор регулярных выражений для анализа кода; реализованы модули нормализации, генерации и модификации кода; разработан REST API для взаимодействия с мобильным приложением; реализован механизм проверки пользовательских решений; выполнена интеграция с мобильным приложением на Kotlin; проведено тестирование разработанного программного средства.

В рамках проекта использован следующий стек технологий: язык программирования Python, фреймворк Django, библиотека Django REST Framework для разработки API, PostgreSQL в качестве системы управления базами данных, Kotlin для мобильного клиента, а также регулярные выражения для анализа и модификации кода.

К особенностям разработанного программного средства можно отнести: автоматическую генерацию заданий на основе пользовательского кода; поддержку нескольких языков программирования (Python, Java, JavaScript); классификацию ошибок по типам и уровням сложности; адаптацию заданий под уровень пользователя; наличие механизма верификации с подсветкой ошибок; модульную архитектуру, обеспечивающую расширяемость системы.

Разработанное решение обладает практической значимостью, так как позволяет автоматизировать процесс создания заданий на отладку кода и

способствует развитию навыков анализа и исправления ошибок у обучающихся.

Перспективы развития проекта включают расширение списка поддерживаемых языков программирования (C++, C#, Go, TypeScript); добавление новых типов ошибок и улучшение алгоритмов генерации; интеграцию с внешними сервисами (например, GitHub); разработку веб-версии приложения; внедрение системы взаимодействия между пользователями; развёртывание системы в облачной инфраструктуре.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Абельсон, Х. Структура и интерпретация компьютерных программ / Х. Абельсон, Дж. Сассман. – Москва : Добросвет, 2006. – 608 с.
2. Ахо, А. Теория синтаксического анализа, перевода и компиляции. Синтаксический анализ / А. Ахо, Дж. Ульман. – Москва : Мир, 1978. – 524 с.
3. Макконнелл, С. Совершенный код / С. Макконнелл. – Санкт-Петербург : Питер, 2019. – 960 с.
4. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. – Москва : Вильямс, 2013. – 464 с.
5. Myers, G. The Art of Software Testing / G. Myers, T. Badgett, T. Thomas. – Hoboken : John Wiley & Sons, 2011. – 256 p.
6. Django REST Framework : документация. – URL: <https://www.django-rest-framework.org> (дата обращения: 18.03.2026).
7. Java : документация Oracle. – URL: <https://docs.oracle.com/en/java/> (дата обращения: 15.03.2026).
8. JavaScript : документация MDN. – URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 15.03.2026).
9. LeetCode : сайт. – URL: <https://leetcode.com> (дата обращения: 10.03.2026).
10. MutPy : инструмент мутационного тестирования для Python. – URL: <https://github.com/mutpy/mutpy> (дата обращения: 20.03.2026).
11. Python : официальный сайт. – URL: <https://www.python.org> (дата обращения: 15.03.2026).
12. Stepik : образовательная платформа. – URL: <https://stepik.org> (дата обращения: 10.03.2026).